

1. To Detect Variable References

Expand Values

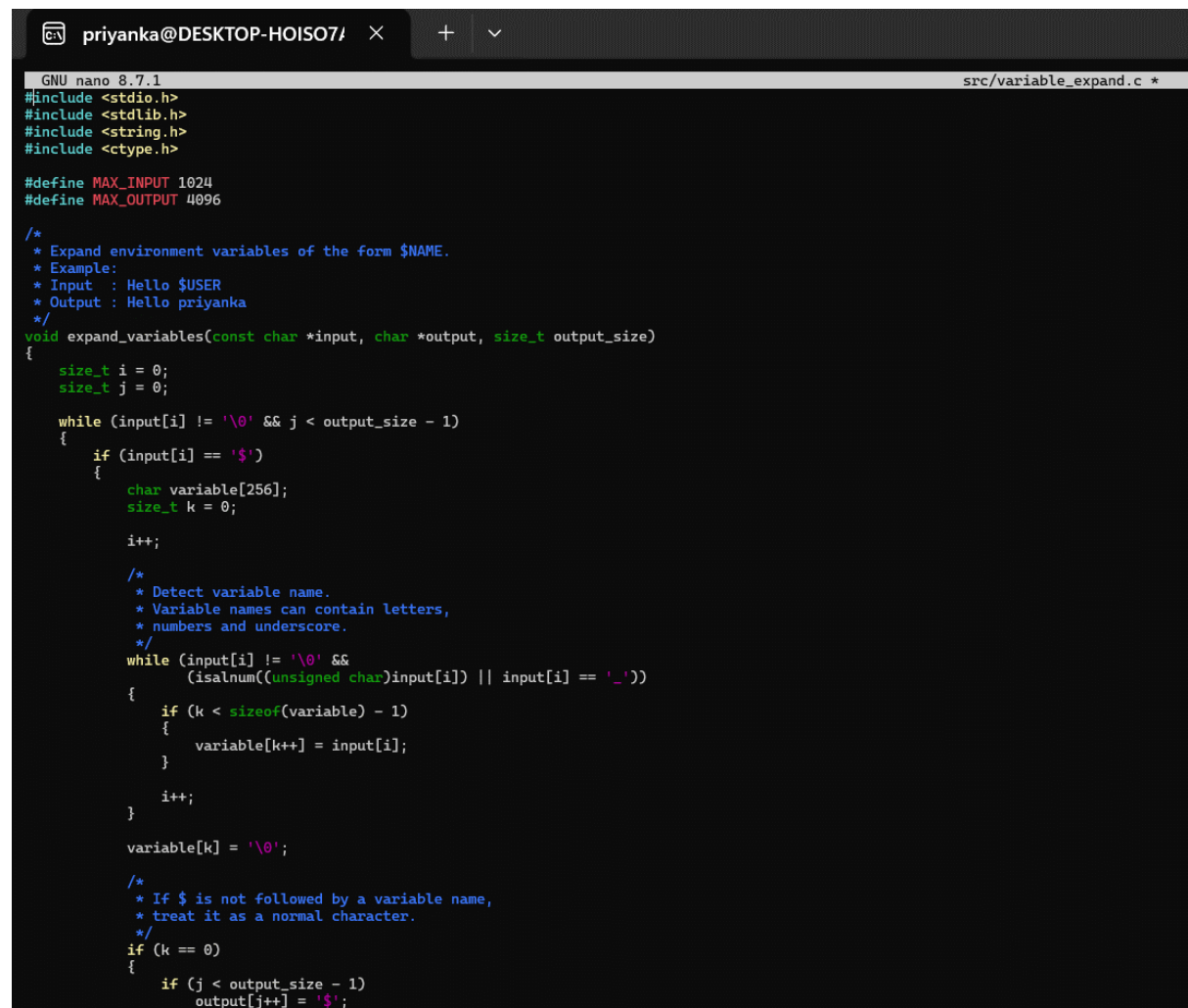
Handle Undefined Variables

Update Tokens

Support Nested Variables

Test Expansion Logic

Code:



```
GNU nano 8.7.1 src/variable_expand.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_OUTPUT 4096

/*
 * Expand environment variables of the form $NAME.
 * Example:
 * Input  : Hello $USER
 * Output : Hello priyanka
 */
void expand_variables(const char *input, char *output, size_t output_size)
{
    size_t i = 0;
    size_t j = 0;

    while (input[i] != '\0' && j < output_size - 1)
    {
        if (input[i] == '$')
        {
            char variable[256];
            size_t k = 0;

            i++;

            /*
             * Detect variable name.
             * Variable names can contain letters,
             * numbers and underscore.
             */
            while (input[i] != '\0' &&
                    (isalnum((unsigned char)input[i]) || input[i] == '_'))
            {
                if (k < sizeof(variable) - 1)
                {
                    variable[k++] = input[i];
                }

                i++;
            }

            variable[k] = '\0';

            /*
             * If $ is not followed by a variable name,
             * treat it as a normal character.
             */
            if (k == 0)
            {
                if (j < output_size - 1)
                    output[j++] = '$';
            }
        }
    }
}
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
        continue;
    }

    /*
     * Retrieve variable value.
     */
    char *value = getenv(variable);

    if (value == NULL)
    {
        /*
         * Undefined variables are replaced
         * with an empty string.
         */
        continue;
    }

    /*
     * Expand the variable value into output.
     */
    size_t value_length = strlen(value);

    if (j + value_length >= output_size)
    {
        value_length = output_size - j - 1;
    }

    memcpy(output + j, value, value_length);
    j += value_length;
}
else
{
    output[j++] = input[i++];
}

output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Variable Expansion Shell\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("expand> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }
    }
}
```

```

    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    expand_variables(input, output, sizeof(output));

    printf("Expanded: %s\n", output);
}

return 0;
}

```

Output:

```

priyanka@DESKTOP-HOISO7A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ pwd
/home/priyanka/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ nano src/variable_expand.c

priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ gcc src/variable_expand.c -o bin/variable_expand
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ls bin
path_resolver  variable_expand  waitpid_demo
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ./bin/variable_expand
Variable Expansion Shell
Type 'exit' to quit.

expand> $USER
Expanded: priyanka
expand> $HOME
Expanded: /home/priyanka
expand> Hello $USER
Expanded: Hello priyanka
expand> User=$USER Home=$HOME
Expanded: User=priyanka Home=/home/priyanka
expand> $ABCXYZ
Expanded:
expand> Hello World
Expanded: Hello World
expand> exit
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ |

```

2. To Identify Built-ins

Create Dispatch Table

Execute In-Process Commands

Handle Invalid Commands

Maintain State

Test Dispatch Logic

Code:

```
priyanka@DESKTOP-HOISO7/  ×  +  ∨
GNU nano 8.7.1 src/builtin_shell.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

/* Function pointer type for built-in commands */
typedef int (*builtin_func)(char **args);

/* ----- Built-in Functions ----- */

int builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        fprintf(stderr, "cd: missing directory\n");
        return 1;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return 1;
    }

    return 1;
}

int builtin_pwd(char **args)
{
    char cwd[1024];

    (void)args;

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("pwd");
        return 1;
    }
}
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
    printf("%s\n", cwd);

    return 1;
}

int builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
        {
            printf(" ");
        }

        i++;
    }

    printf("\n");

    return 1;
}

int builtin_help(char **args)
{
    (void)args;

    printf("\nBuilt-in commands:\n");
    printf("  cd <directory>  Change directory\n");
    printf("  pwd             Display current directory\n");
    printf("  echo <text>      Display text\n");
    printf("  help            Display this help message\n");
    printf("  exit            Exit the shell\n\n");

    return 1;
}
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
int builtin_exit(char **args)
{
    (void)args;

    return 0;
}

/* ----- Dispatch Table ----- */

typedef struct
{
    const char *name;
    builtin_func function;
} Builtin;

/*
 * Create dispatch table.
 */
Builtin builtins[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

#define BUILTIN_COUNT (sizeof(builtins) / sizeof(builtins[0]))

/* ----- Command Parsing ----- */

int parse_command(char *input, char **args)
{
    int count = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
    }
}
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    return count;
}

/* ----- Dispatch Function ----- */

int execute_builtin(char **args)
{
    if (args[0] == NULL)
    {
        return 1;
    }

    for (size_t i = 0; i < BUILTIN_COUNT; i++)
    {
        if (strcmp(args[0], builtins[i].name) == 0)
        {
            return builtins[i].function(args);
        }
    }

    printf("Invalid command: %s\\n", args[0]);

    return 1;
}

/* ----- Main Shell ----- */

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Built-in Command Shell\\n");
    printf("Type 'help' to see available commands.\\n\\n");
    |
```

```

while (1)
{
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    parse_command(input, args);

    if (args[0] == NULL)
    {
        continue;
    }

    if (execute_builtin(args) == 0)
    {
        break;
    }
}

printf("Shell terminated.\n");

return 0;
}

```

|

^G Help

^O Write Out

^F Where Is

^K Cut

Output:

```
priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4$ nano src/builtin_shell.c

priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4$ gcc src/builtin_shell.c -o bin/builtin_shell
priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4$ ls bin
builtin_shell  path_resolver  variable_expand  waitpid_demo
priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4$ ./bin/builtin_shell
Built-in Command Shell
Type 'help' to see available commands.

myshell> help

Built-in commands:
cd <directory>  Change directory
pwd             Display current directory
echo <text>     Display text
help           Display this help message
exit           Exit the shell

myshell> pwd
/home/priyanka/OS_Skill/Week-4
myshell> echo Hello World
Hello World
myshell> pwd
/home/priyanka/OS_Skill/Week-4
myshell> hello123
Invalid command: hello123
myshell> Hello
Invalid command: Hello
myshell> exit
Shell terminated.
priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4$ |
```